From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging

Yuling Shi
Shanghai Jiao Tong University
Shanghai, China
yuling.shi@sjtu.edu.cn

Songsong Wang
University of California, Davis
Davis, USA
ssswang@formerstudents.ucdavis.edu

Chengcheng Wan
East China Normal University
Shanghai, China
ccwan@sei.ecnu.edu.cn

Min Wang
University of Pennsylvania
Philadelphia, USA
minyun@seas.upenn.edu

Xiaodong Gu*
Shanghai Jiao Tong University
Shanghai, China
xiaodong.gu@sjtu.edu.cn

arXiv:2410.01215v4 [cs.CL] 22 Nov 2025

Abstract

While large language models have made significant strides in code generation, the pass rate of the generated code is bottlenecked on subtle errors, often requiring human intervention to pass tests, especially for complex problems. Existing LLM-based debugging systems treat generated programs as holistic units, failing to address bugs at multiple levels of granularity, from low-level syntax errors to high-level algorithmic flaws. In this paper, we introduce *Multi-Granularity Debugger* (MGDebugger), a hierarchical code debugger that isolates, identifies, and resolves bugs at various levels of granularity. MGDebugger decomposes problematic code into a hierarchical tree structure of subfunctions, with each level representing a particular granularity of error. During debugging, it analyzes each subfunction and iteratively resolves bugs in a bottom-up manner. To effectively test each subfunction, we propose an LLM-simulated Python executor, which traces code execution and tracks important variable states to pinpoint errors accurately. Extensive experiments with both open-source and commercial LLMs demonstrate that MGDebugger significantly outperforms existing debugging systems, achieving up to 18.9% improvement in accuracy over seed generations in HumanEval and a 97.6% repair success rate in HumanEvalFix. Furthermore, MGDebugger effectively generalizes to real-world software defects, fixing 129 bugs in Defects4J v1.2 and 64 bugs in Defects4J v2.0, outperforming state-of-the-art program repair approaches by 13.2% and 8.5% respectively. These results demonstrate MGDebugger's robustness and effectiveness across different scenarios, establishing it as a powerful tool for closing the last mile of code generation.[1]

ACM Reference Format:
Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. 2026. From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3773255

*Corresponding author.
[1]Code and data available at https://github.com/YerbaPage/MGDebugger

This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3773255

1 Introduction

Large language models (LLMs) such as GPT-4 [48], LLaMA [56], and DeepSeek-Coder [1] have made significant advances in AI-assisted coding tasks [7, 25, 34, 52]. Trained on vast corpora of text and code, LLMs can understand and generate code snippets for various programming tasks, ranging from simple data structures to complex algorithmic problems [34, 57]. These models have demonstrated proficiency in tasks such as code completion, bug detection, and even tackling competitive programming challenges [7, 59, 68].

While the code generated by large language models generally meets the requirements, it often contains critical errors that require human intervention to pass tests [12, 37]. This has gradually led to a new development paradigm: large models generate the code, while humans fix it. Therefore, the "last mile", as well as the most crucial step, of code generation is how to efficiently repair the code generated by large models.

Numerous efforts have been made to debug LLM-generated code. The most popular way is to reuse the LLM generator to debug the generated code with the feedback from test case execution [9, 16, 75]. While these methods increase the pass rates, they treat the erroneous program as a holistic set of statements (i.e., as a single unit without considering the internal structure or modularity) [9, 10, 53, 75] regardless of the varying types and levels of failures. Failures of test cases arise from different levels of factors, from low-level syntactic errors to high-level algorithmic flaws. A holistic treatment overlooks the internal structure of the code and limits the effectiveness of the debugging systems, especially when dealing with complex programs that need debugging across different modules [55, 67].

In this paper, we introduce Multi-granularity Debugger (MGDebugger), a novel debugging method for LLM-generated code. Instead of treating entire functions as single units, MGDebugger employs a hierarchical, bottom-up strategy to systematically debug code. It begins by decomposing the code into a tree structure of sub-functions, allowing for the isolation of semantic units for independent debugging. Each sub-function is debugged progressively, starting with the most granular ones and working upward to higher-level compositions until the entire code is repaired. To effectively test and debug each subfunction, MGDebugger generates test cases derived

from the public test cases of the main function. Then, it employs an LLM-based execution simulator to track changes in key variables, facilitating precise and flexible error identification based on the failed test cases. Through debugging at multiple levels of granularity from the bottom up in a recursive manner, MGDebugger can uncover and rectify bugs that traditional holistic debugging methods might overlook.

Extensive experiments across multiple models and benchmarks demonstrate that MGDebugger significantly outperforms existing debugging methods. Using open-source models like DeepSeek-Coder-V2-Lite, CodeQwen1.5, and Codestral, MGDebugger elevates accuracy from 75.6% to 94.5% on HumanEval [7] and achieves a remarkable 97.6% repair success rate on HumanEvalFix [44]. The approach also generalizes well to state-of-the-art proprietary models, improving GPT-4o and Claude 3.5 Sonnet's performance by 6.1% on HumanEval, reaching accuracies of 96.3% and 95.7% respectively.

Our comprehensive ablation studies confirm the vital role of each component in MGDebugger, with the hierarchical debugging strategy being the most critical element. Removing this component leads to a significant drop in repair success rate from 76.3% to 52.6% on HumanEval, highlighting its importance for systematic bug identification and resolution. Detailed analyses across different bug categories reveal MGDebugger's exceptional performance in handling various error types, achieving 100% success rates in fixing missing logic, excess logic, operator misuse, variable misuse, and function misuse bugs with DeepSeek-Coder. Furthermore, MGDebugger maintains robust performance across code snippets of varying lengths and complexities, consistently outperforming baseline methods especially for longer, more complex code where other approaches typically struggle.

While MGDebugger is primarily designed to fix LLM-generated code and close the last mile of code generation, we also evaluate its generalizability to real-world software defects. When applied to the Defects4J dataset [25], MGDebugger successfully repairs 129 bugs in Defects4J v1.2 and 64 bugs in Defects4J v2.0, outperforming specialized program repair tools. This demonstrates that the hierarchical debugging principles with test case generation and LLM-simulated execution of MGDebugger extend effectively beyond LLM-generated code to human-written software with complex project structures.

The main contributions of this work can be summarized as:

- We introduce MGDebugger, a novel hierarchical debugging method to repair LLM-generated code at multiple levels of granularity.
- We propose an LLM-simulated execution approach that enables precise bug identification by tracing code execution and tracking variable states within subfunctions, eliminating the need for external debuggers.
- We demonstrate MGDebugger's effectiveness through comprehensive experiments across multiple models and benchmarks, showing significant improvements over existing methods with up to 18.9% accuracy gains on HumanEval and 97.6% repair success rate on HumanEvalFix.
- We establish MGDebugger's generalizability by successfully applying it to real-world software defects in Defects4J, outperforming specialized program repair tools and demonstrating its potential as a unified debugging framework for both AI-generated and human-written code.

2 Methodology

2.1 Overview

We present MGDebugger, a novel bottom-up hierarchical debugging method for repairing LLM-generated code. The overall workflow of MGDebugger transforming holistic debugging into hierarchical subfunction debugging is illustrated in Figure 1, while the detailed debugging process for each subfunction is depicted in Figure 2.

As shown in Figure 1, MGDebugger begins with *Hierarchical Code Decomposition (Section 2.2)*, which decomposes the input buggy code into a hierarchical structure of subfunctions. This enables systematic identification and resolution of bugs at various levels of granularity. For each subfunction, MGDebugger *Generates Test Cases for Subfunctions (Section 2.3)*, deriving private test cases from public test cases of the main function, as illustrated in Figure 2. MGDebugger then executes these test cases and *Debugs Subfunction with LLM-Simulated Execution (Section 2.4)*. The LLM simulates step-by-step code execution for failed test cases, monitoring critical variables and state changes to pinpoint the cause of errors. Once a subfunction has been fixed, MGDebugger updates it in the hierarchical structure and propagates the changes to dependent functions through *Bottom-up Debugging (Section 2.5)*. This hierarchical debugging approach not only tackles different types of bugs at various levels of abstraction but also guarantees a cohesive and systematic debugging process throughout the entire code structure.

2.2 Hierarchical Code Decomposition

Modularizing and decomposing complex code into smaller helper subfunctions has been proven to be helpful especially for large functions that are difficult to understand [21, 66]. To enable hierarchical debugging, we need to transform the input code into a tree-like structure of subfunctions.

Specifically, given an LLM-generated function f, we decompose it into a hierarchical structure of subfunctions denoted as $(f_1, ..., f_n)$. We formally define a "subfunction" as a contiguous sequence of statements $S = [s_1, s_2, ..., s_k]$ within function f, such that S represents a logically coherent and minimal unit of functionality that can be abstracted as a function $f_i(x)$ with well-defined inputs and outputs. These subfunctions can be organized as a tree $f_{\text{root}} = \text{TREE}(f_{\text{root}}, \text{CHILD}(f_{\text{root}}))$, where f_{root} represents the main function and $\text{CHILD}(f)$ denotes the set of subfunctions directly called by f. We leverage an LLM for the decomposition, adhering to three principles: (1) each subfunction represents the minimal reusable unit of code with a specific purpose, (2) higher-level functions call lower-level functions to achieve complex functionality, and (3) the overall structure facilitates isolated testing and debugging. As illustrated in Figure 1, the resulting tree-like structure allows us to isolate logical units of the code, enabling more focused debugging efforts across different levels of granularity [19, 60]. The prompt template for hierarchical decomposition is shown below:

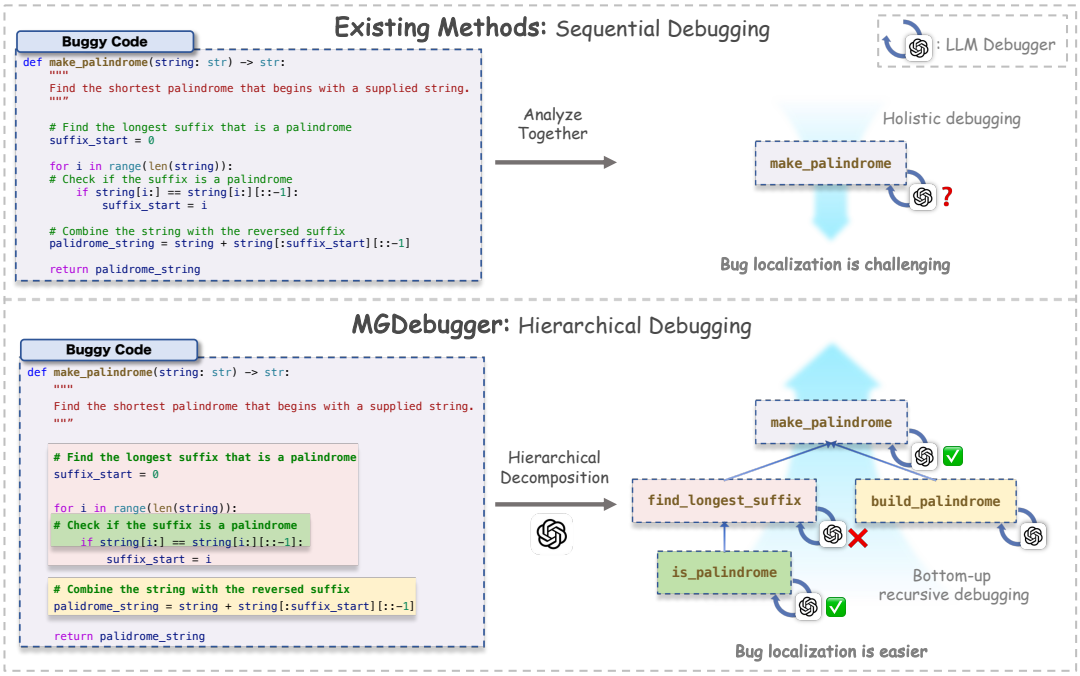


Figure 1: Workflow of MGDebugger compared to existing methods.

Prompt Template for Hierarchical Decomposition

Convert the following Python code into a tree-style hierarchical structure with multiple levels of sub-functions. Each significant step or logical block should be its own function, and functions can call other sub-functions. Ensure that the main function calls these sub-functions in the correct order, creating a tree-like structure.
Original Code:
{code}
Instruction:
Please first analyze the codes step by step, and then provide the converted code in a Python code block. When providing the final converted code, make sure to include all the functions in a flattened format, where each function is defined separately.

2.3 Generating Test Cases for Subfunctions

Having obtained the hierarchy of subfunctions, we aim to verify the correctness of each subfunction. For this purpose, we generate test cases for each subfunction leveraging automatic unit test generation techniques [38, 50, 58]. For each subfunction $f_i \in f_{\text{root}}$, we generate a set of test cases $\mathcal{T}_i$. Following the problem settings from Chen et al. [9] and Zhong et al. [75], we assume that the public test cases for the main function $\mathcal{T}_{\text{pub}}$ have been provided, which is common in most code generation benchmarks [7, 14, 44][2]. We can leverage these test cases to derive a set of corresponding test cases for each subfunction.

We employ the same LLM for the test case generation. For each $f_i \in f_{\text{root}}$. The LLM is now prompted to perform the following steps: (1) analyze how the subfunction is used within the main function and how it contributes to the expected outputs in the public test cases; (2) for each public test case, reason through the overall code structure step by step to figure out the input and expected output for the subfunction. This approach ensures that the generated test cases are not only reflective of the subfunction's intended functionality but also contextualized within the constraints provided by the public test cases, enhancing the robustness and relevance of the test cases. The prompt template for test case generation is shown below:

Prompt Template for Test Case Generation

SYSTEM_PROMPT:
You are an AI assistant specialized in analyzing Python functions and generating test cases.
USER_PROMPT:
Full Code:
{full_code}
Public Test Cases for the Main Function:
{public_test_cases}
Instruction:
Please analyze how the {function_name} function is used within the main function and how it contributes to the expected outputs in the gold test cases. For each test case, you should analyze step-by-step based on both the input and the expected output of the main function, and then provide the corresponding input and expected output for the {function_name} function. Ensure that the generated test cases are consistent with the behavior expected in the public test cases.

2.4 Debugging Subfunctions with LLM-simulated Execution

With the generated test cases, we debug each subfunction by running them on the test case inputs, obtaining the results, and comparing these results against the expected outcomes in the test cases. When a failed test case is identified, we fix the corresponding subfunction and produce a corrected version.

One straightforward way to implement this process is to use an external Python executor to monitor runtime variable values [75].

[2]Otherwise, we can use LLM-generated test cases instead.

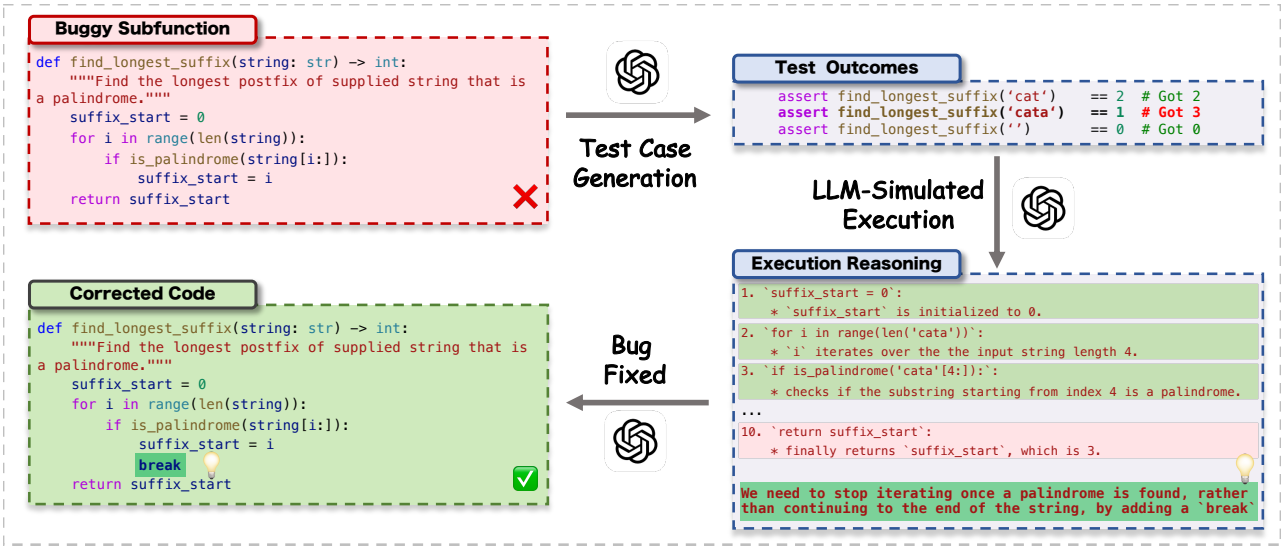


Figure 2: Illustration of the subfunction debugging process in MGDebugger.

However, when debugging high-level functions, tracking variable values within lower-level subfunctions is often unnecessary, as their correctness is ensured by the bottom-up debugging methodology. Furthermore, directly collecting all execution traces from the external debugger can add unnecessary overhead and complexity to the process.

Inspired by the methodology in Li et al. [33], we propose an LLM-simulated code executor, which prompts LLM to act as a code interpreter and track the code execution. We choose LLM-simulated execution over real execution for several key reasons [29, 45, 73]: (1) Many code snippets require specific environments, hardware, or dependencies that are difficult to set up or may not be available (e.g., embedded systems, special libraries), (2) Some code cannot be executed due to compilation errors or security concerns, necessitating sandboxing or virtual execution. As shown in Figure 2, we request the LLM to simulate the execution process, reasoning about key variables and their states at each step, and thoroughly analyzing the failed test cases. This eliminates the need for an external debugger, offering a more flexible and efficient debugging solution. In addition, the LLM can accurately identify where errors occur and grasp their surrounding context. The prompt template for debugging subfunction is shown below:

Prompt Template for Debugging Subfunction

SYSTEM_PROMPT:
You are an AI assistant helping to debug Python functions.
USER_PROMPT:
Debug the following Python function. The function is not passing all test cases. Analyze the code, identify the bug, and provide a fixed version of the function.
Function Code:
{function_code}
Test Case Results:
{test_case_results}
Instruction:
Please try to work as a Python interpreter to execute the code step-by-step. Identify the change of each variable as you "run" the code line-by-line. Based on the execution trace, try to identify the bug and provide the final fixed code in a Python code block.

Algorithm 1 MGDebugger: Bottom-up Recursive Debugging

Input: f: Input LLM-generated function; $\mathcal{T}_{\text{pub}}$: Public test cases.
Output: f': Debugged f.

```
function MGDebugger(f, T_pub)
    if f has subfunctions {f_1, ..., f_n} then
        for f_i ∈ f do                          ▷ Depth-first traversal
            f'_i ← MGDebugger(f_i, T_pub)       ▷ Recursive debugging
            f_i = f'_i                          ▷ Replace f_i with the debugged version
        end for
    end if
    T_f ← GenTest(f, T_pub)                     ▷ Generate test cases for f
    R_f ← Exec(f, T_f)                          ▷ Execute test cases for f
    if pass (R_f, T_f) then
        return f                                ▷ Correct function; keep as is
    else
        f' ← Debug(f, T_f, R_f)                 ▷ Debug function f based on test results R_f
        return f'                               ▷ Return the corrected code
    end if
end function
```

2.5 Bottom-up Debugging

Having introduced code decomposition and the debugging process for each subfunction, we now outline the overall debugging workflow.

We initiate the process by calling MGDebugger on the main function with the decomposed code f_{root} and the set of public test cases $\mathcal{T}_{\text{pub}}$. MGDebugger traverses the hierarchical structure in a depth-first manner, recursively debugging each subfunction before moving on to the higher-level functions. For each specific subfunction, MGDebugger generates relevant test cases and debugs the function based on the results. When a fix is identified, MGDebugger updates the function and propagates the changes to the

dependent functions. This recursive, bottom-up strategy systematically addresses bugs by beginning with the most granular levels and progressively advancing through the function hierarchy. This method accommodates various types of bugs at different abstraction levels, from low-level syntax errors to high-level logical flaws, by focusing on one level of the hierarchy at a time and building up the corrected code in a structured manner. The detailed algorithm is presented in Algorithm 1.

3 Experiments

We conduct experiments to evaluate the effectiveness of MGDebugger, aiming to answer the following research questions:

- **RQ1:** How effective is MGDebugger in debugging LLM-generated code?
- **RQ2:** To what extent do individual components of MGDebugger contribute to its overall performance?
- **RQ3:** How does MGDebugger perform in fixing bugs across different categories?
- **RQ4:** How consistent is MGDebugger's performance across different debugging scenarios?
- **RQ5:** How will MGDebugger generalize to fixing real-world software defects?

3.1 Setup

Models We select three state-of-the-art open-source LLMs ranging from 7B to 22B parameters as backbones for code generation and debugging in the experiments: CodeQwen1.5 (7B) [54], DeepSeek-Coder-V2-Lite (16B) [1], and Codestral (22B) [42]. To validate the generalizability of MGDebugger, stronger general-prupose models like GPT-4o [48], Claude 3.5 Sonnet [2], and LLaMA 3.1 70B [56] are also included in the Experiments.

Datasets We conduct experiments on three widely used benchmarks for LLM-based code generation and debugging. HumanEval [7] and MBPP [3] are two widely used benchmarks for evaluating code generation systems with 164 and 500 problems, respectively. The HumanEvalFix dataset [44] consists of 164 buggy functions with six different bug categories: value misuse, missing logic, excess logic, operator misuse, variable misuse, and function misuse.

Metrics We adopt two metrics to evaluate our method: 1) *Accuracy* [9, 75], which measures the overall proportion of correct code samples among all generated code samples after debugging. A code is correct iff it passes all private test cases assigned to it. 2) *Repair Success Rate* (RSR) [64], which refers to the proportion of fixed code samples to the total number of buggy code samples. The former metric indicates to which extent MGDebugger can improve the code generation systems, and the latter metric directly reflects the effectiveness of MGDebugger in fixing bugs from LLM-generated code.

Baselines We compare MGDebugger with eight state-of-the-art methods for debugging LLM-generated code. 0) *No-Debugging* baseline refers to directly using the LLM to generate code without applying any debugging method, following [9, 75]. This baseline creates the initial set of seed programs, the subset of buggy programs then become targets for various debugging methods. 1) *Simple Feedback* is a basic baseline that informs the LLM that the code is incorrect and asks it to fix the issue. 2) *Self-Edit* [69] prompts the LLM to edit the code based on the execution results of the test cases. 3) *Self-Debugging* [9] has two variants: *Self-Debugging (Expl.)* prompts the LLM to explain the generated code line-by-line, while *Self-Debugging (Trace)* asks the LLM to dry run the code for debugging. 4) *LDB* [75] segments the code into basic *blocks*, *functions* or *lines*, and tracks variable values during runtime after each block to verify correctness against the task description. 5) *Reflexion* [53] asks the LLM to reflect on the previous code given execution results and uses a memory buffer to enable iterative refinement.

Implementation Details We generate seed programs for HumanEval and MBPP using the BigCode Evaluation Harness framework[3]. The specific versions of models used in our experiments are DeepSeek-Coder-V2-Lite-Instruct, CodeQwen1.5-7B-Chat, and Codestral-22B-v0.1. All experiments are conducted on NVIDIA A100 GPUs with 80GB memory. During debugging, we use the vLLM engine[4] to serve the LLMs, setting the maximum token length according to each LLM's max length. Following Zhong et al. [75], we limit the maximum number of debugging iterations to 10 for all methods on HumanEval, MBPP, and HumanEvalFix. To obtain visible test cases for HumanEval and HumanEvalFix, we extract the given visible test cases from the task description. For MBPP, we use the first test case of each problem as the visible test case and use the rest as hidden test cases, in line with the settings referenced from Chen et al. [9] and Zhong et al. [75]. Additionally, the sampling temperature that controls the randomness of LLM decoding is set to 0.8 in MGDebugger. For baseline methods that incorporate external feedback from the environments, we follow [75] to execute the complete test code to collect runtime feedback.

3.2 Main Results (RQ1)

The results in Table 1 show that MGDebugger consistently outperforms existing approaches across all models and datasets. Specifically, MGDebugger achieves the highest accuracy improvements, with gains of +15.3% to +18.9% on HumanEval and +11.4% to +13.4% on MBPP. These improvements are particularly notable when compared to baseline methods such as Self-Debugging (Expl.) and Reflexion, which also incorporate external feedback but exhibit lower gains in accuracy and RSR. The strong results across models of varying sizes highlight the adaptability of MGDebugger to different LLM architectures.

Moreover, MGDebugger demonstrates remarkable debugging capabilities, particularly with DeepSeek-Coder-V2-Lite (16B) and Codestral (22B), where it achieves an accuracy of 94.5% on the HumanEval dataset, the highest score among all methods. This is especially impressive considering that MGDebugger operates in a zero-shot setting without task-specific retraining. This result illustrates the inherent debugging ability of larger LLMs with MGDebugger. Additionally, the method's performance on MBPP, achieving an RSR of up to 41.1% with smaller models like CodeQwen1.5 (7B), further underscores its robustness. Statistical analysis using the Wilcoxon signed-rank test with 10 repetitions confirms that these improvements are significant ($p < 0.005$). In general, these results validate MGDebugger as a highly effective and scalable debugging method for LLM-generated code.

[3] https://github.com/bigcode-project/bigcode-evaluation-harness
[4] https://github.com/vllm-project/vllm

Table 1: Main results.

Method	Dataset			
	HumanEval		MBPP	
	Acc. (%)	RSR (%)	Acc. (%)	RSR (%)
DeepSeek-Coder-V2-Lite				
No-Debugging	76.8	–	67.2	–
Simple Feedback	82.3 (+5.5)	23.7	69.4 (+2.2)	6.7
Self-Edit	82.9 (+6.1)	26.3	71.2 (+4.0)	12.2
LDB (Block)	84.1 (+7.3)	31.6	74.0 (+6.8)	20.7
LDB (Line)	82.3 (+5.5)	23.7	71.8 (+4.6)	14.0
LDB (Function)	81.7 (+4.9)	21.1	72.6 (+5.3)	16.5
Self-Debugging (Expl.)	87.2 (+10.4)	44.7	73.4 (+6.2)	18.9
Self-Debugging (Trace)	86.0 (+9.2)	39.5	72.6 (+5.3)	16.5
Reflexion	90.9 (+14.1)	60.5	76.6 (+9.4)	28.7
MGDebugger	**94.5 (+17.7)**	**76.3**	**80.0 (+12.8)**	**39.0**
CodeQwen1.5				
No-Debugging	76.2	–	67.4	–
Simple Feedback	85.4 (+9.2)	38.5	74.0 (+6.6)	20.2
Self-Edit	84.1 (+7.9)	33.3	75.0 (+7.6)	23.3
LDB (Block)	79.3 (+3.1)	12.8	72.8 (+5.4)	16.6
LDB (Line)	79.9 (+3.7)	15.4	72.6 (+5.2)	16.0
LDB (Function)	80.5 (+4.3)	17.9	72.8 (+5.4)	16.6
Self-Debugging (Expl.)	87.8 (+11.6)	48.7	77.4 (+10.0)	30.7
Self-Debugging (Trace)	84.8 (+8.6)	35.9	76.8 (+9.4)	28.8
Reflexion	87.8 (+11.6)	48.7	78.6 (+11.2)	34.4
MGDebugger	**91.5 (+15.3)**	**64.1**	**80.8 (+13.4)**	**41.1**
Codestral				
No-Debugging	75.6	–	65.4	–
Simple Feedback	88.4 (+12.8)	52.5	71.6 (+6.2)	17.9
Self-Edit	86.0 (+10.4)	42.5	75.8 (+10.4)	30.0
LDB (Block)	83.5 (+7.9)	32.5	72.2 (+6.8)	19.7
LDB (Line)	83.5 (+7.9)	32.5	71.8 (+6.4)	18.5
LDB (Function)	82.3 (+6.7)	27.5	72.0 (+6.6)	19.1
Self-Debugging (Expl.)	89.6 (+14.0)	57.5	76.4 (+11.0)	31.8
Self-Debugging (trace)	84.1 (+8.5)	35.0	73.6 (+8.2)	23.7
Reflexion	86.6 (+11.0)	45.0	75.2 (+9.8)	28.3
MGDebugger	**94.5 (+18.9)**	**77.5**	**76.8 (+11.4)**	**32.9**

Table 2: Results (Acc.) on HumanEval with advanced models.

Method	Model		
	GPT-4o	Claude 3.5 Sonnet	LLaMA 3.1 70B
No-Debugging	90.2	89.6	79.3
Self-Debugging (Expl.)	93.3 (+3.1)	93.3 (+3.7)	86.6 (+7.3)
Reflexion	94.5 (+4.3)	93.9 (+4.3)	89.6 (+10.4)
MGDebugger	**96.3 (+6.1)**	**95.7 (+6.1)**	**92.7 (+13.4)**

To further validate the effectiveness and generalizability of MGDebugger across a wider spectrum of language models, we extended our evaluation to include recent advanced LLMs with strong natural language capabilities, including GPT-4o, Claude 3.5 Sonnet, and LLaMA 3.1 70B. We compare MGDebugger with the most competitive methods, Self-Debugging (Expl.) and Reflexion, on the HumanEval dataset.

Table 3: Results of ablation study.

Method	HumanEval		MBPP	
	Acc. (%)	RSR (%)	Acc. (%)	RSR (%)
MGDebugger	**94.5** (+17.7)	**76.3**	**80.0** (+12.8)	**39.0**
- w/o Hierarchical Debugging	89.0 (+12.2)	52.6	78.2 (+11.0)	33.5
- w/o Simulated Execution	90.2 (+13.4)	61.3	79.2 (+12.0)	36.6
- w/o Test Case Generation	90.9 (+14.1)	60.5	79.2 (+12.0)	36.6
- w/ Real Execution	92.7 (+15.9)	47.4	79.0 (+11.8)	36.0
No-Debugging	76.8	–	67.2	–

The results in Table 2 demonstrate that MGDebugger continues to outperform existing debugging approaches even on state-of-the-art proprietary models like GPT-4o and Claude 3.5 Sonnet, which already possess strong zero-shot code generation capabilities. With these advanced models, MGDebugger achieves impressive accuracy improvements of +6.1% for both GPT-4o and Claude 3.5 Sonnet, pushing their overall accuracy to 96.3% and 95.7% respectively on HumanEval. This represents a substantial improvement over strong baselines like Self-Debugging and Reflexion. Notably, the results on LLaMA 3.1 70B demonstrate that MGDebugger provides even greater benefits for open-source models with enhanced natural language capabilities. The method yields a remarkable +13.4% improvement in accuracy, bringing LLaMA 3.1 70B's performance to 92.7%.

These findings strengthen our previous conclusions about the adaptability of MGDebugger across different model architectures. We also reveal an interesting trend: while state-of-the-art models like GPT-4o and Claude 3.5 Sonnet start with higher baseline performance, MGDebugger still provides substantial improvements, suggesting that even the most advanced LLMs benefit significantly from structured debugging approaches. This highlights the potential for MGDebugger as a general enhancement method that can complement the evolving landscape of AI coding assistants.

3.3 Ablation Study (RQ2)

To understand the contribution of each component in MGDebugger and validate our design choices, we conduct an ablation study by systematically removing key components of our method: hierarchical code decomposition, LLM-simulated execution, and test case generation for subfunction debugging. Additionally, we explore the impact of replacing simulated execution with real execution traces collected with LDB [75] to further validate our design choice. Each variant is evaluated on both the HumanEval and MBPP datasets using the DeepSeek-Coder-V2-Lite model.

As shown in Table 3, each component of MGDebugger plays a crucial role in the overall effectiveness of the method. Among them, the hierarchical debugging strategy is the most impactful component. By ablating this strategy, the repair success rate drops significantly from 76.3% to 52.6% on HumanEval and from 39.0% to 33.5% on MBPP. This result highlights the importance of the hierarchical approach in systematically identifying and fixing bugs at different granularity levels. Additionally, the LLM-simulated execution and test case generation for subfunctions also facilitate debugging the decomposed code, yielding substantial improvements in accuracy and repair success rates. These results underscore the

Table 4: Performance (RSR) on different bug categories in HumanEvalFix. The best and second-best scores are highlighted in bold and underline, respectively.

Method	Value	Missing Logic	Excess Logic	Operator	Variable	Function	Overall
DeepSeek-Coder-V2-Lite							
Simple Feedback	84.9	96.0	80.7	78.3	86.4	87.5	85.4
Self-Edit	78.8	92.0	80.7	82.6	84.1	62.5	82.3
LDB (Block)	69.7	96.0	74.2	87.0	86.4	62.5	81.1
LDB (Line)	63.6	84.0	67.7	73.9	84.1	62.5	74.4
LDB (Function)	69.7	88.0	71.0	87.0	77.3	62.5	76.8
Self-Debugging (Expl.)	66.7	80.0	64.5	78.3	86.4	50.0	74.4
Self-Debugging (Trace)	81.8	88.0	71.0	78.3	79.6	75.0	79.3
Reflexion	**90.9**	**100.0**	90.3	91.3	86.4	**100.0**	91.5
MGDebugger	87.9	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**97.6**
CodeQwen1.5							
Simple Feedback	**81.8**	92.0	87.1	69.6	81.8	87.5	82.9
Self-Edit	72.7	92.0	80.7	65.2	**86.4**	87.5	80.5
LDB (Block)	36.4	72.0	51.6	60.9	63.6	62.5	56.7
LDB (Line)	36.4	76.0	45.2	56.5	54.6	50.0	52.4
LDB (Function)	27.3	60.0	51.6	56.5	59.1	62.5	51.2
Self-Debugging (Expl.)	69.7	92.0	**90.3**	69.6	77.3	62.5	78.7
Self-Debugging (Trace)	72.7	72.0	80.6	69.6	70.5	75.0	73.2
Reflexion	66.7	88.0	80.6	91.3	**86.4**	75.0	81.7
MGDebugger	78.8	**96.0**	87.1	**95.7**	84.1	**100.0**	**87.8**
Codestral							
Simple Feedback	75.8	92.0	67.7	82.6	84.1	62.5	79.3
Self-Edit	78.8	**100.0**	80.7	87.0	84.1	**87.5**	85.4
LDB (Block)	66.7	92.0	67.7	82.6	81.8	**87.5**	78.1
LDB (Line)	63.6	92.0	64.5	82.6	81.8	75.0	76.2
LDB (Function)	57.6	88.0	67.7	**91.3**	75.0	75.0	74.4
Self-Debugging (Expl.)	75.8	96.0	83.9	87.0	90.9	**87.5**	86.6
Self-Debugging (Trace)	57.6	84.0	64.5	73.9	81.8	75.0	72.6
Reflexion	69.7	88.0	61.3	82.6	88.6	75.0	78.0
MGDebugger	**87.9**	**100.0**	**87.1**	82.6	**95.5**	75.0	**90.2**

effectiveness of MGDebugger's design choices and the importance of its hierarchical debugging strategy.

Furthermore, we observe that replacing simulated execution with real execution leads to a noticeable drop in performance, particularly in repair success rate (from 76.3% to 47.4% on HumanEval and from 39.0% to 36.0% on MBPP). This suggests that real execution traces, which include all detailed variable changes and execution steps, may introduce noise that distracts the model during debugging. In contrast, simulated execution allows MGDebugger to focus on key variables and express debugging steps in a more reasoning-oriented manner, leading to better code understanding and more accurate repairs. Furthermore, simulated execution offers greater flexibility for extension to complex programming environments where execution environments may not be easily accessible [45]. This makes MGDebugger more adaptable to a wider range of scenarios.

3.4 Debugging Different Types of Bugs (RQ3)

To assess the versatility and effectiveness of MGDebugger across various bug categories, we carry out experiments using the HumanEvalFix dataset, which is specifically designed to evaluate code debugging performance. The dataset involves six distinct bug categories: value misuse, missing logic, excess logic, operator misuse, variable misuse, and function misuse, allowing us to examine how effectively MGDebugger addresses different types of programming errors compared to existing methods.

Table 4 presents the RSRs across various bug categories with. We observe that MGDebugger consistently outperforms other methods with significantly higher overall accuracies. Specifically, MGDebugger achieves a remarkable repair success rate of 97.6% using DeepSeek-Coder, with 100% success rates in all bug categories except for value misuse. This is particularly notable given the complexity and diversity of the bugs in the dataset. Statistical analysis using the Wilcoxon signed-rank test with 10 repetitions confirms that these improvements are significant ($p < 0.005$). This highlights the effectiveness of the hierarchical debugging strategy.

Table 5: Number of bugs fixed in Defects4J.

Method	Chart	Closure	Lang	Math	Mockito	Time	V1.2	V2.0
CodexRepair	9	30	22	29	6	3	99	31
GAMMA	11	24	16	25	3	3	82	45
ThinkRepair	11	31	19	27	6	4	98	47
ContrastRepair	12	32	19	30	8	2	103	40
Mulpor	13	26	19	23	7	4	92	**59**
ChatRepair	**15**	37	21	32	6	3	114	48
MGDebugger	13	**40**	**27**	**40**	**9**	**5**	**129**	**64**

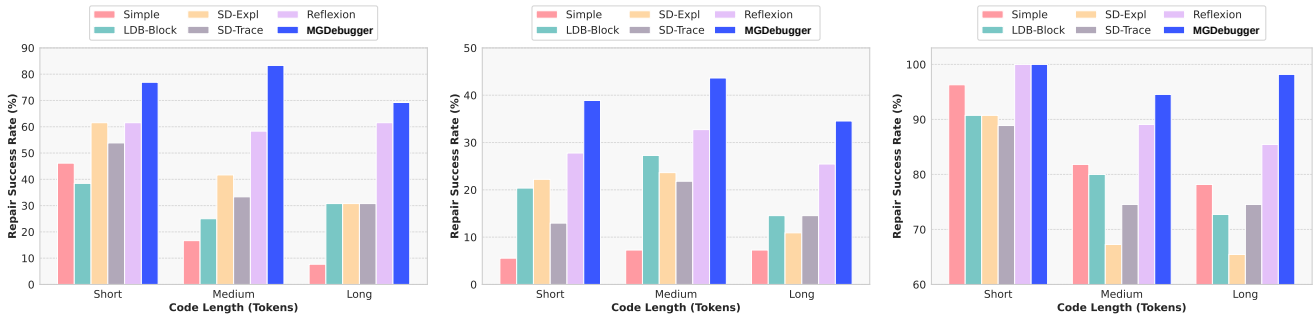
Looking into details of different bug categories, MGDebugger shows a strong advantage in debugging bottom-level bugs, such as missing logic and excess logic. Missing logic refers to situations where essential code is omitted, preventing the solution from functioning correctly. Excess logic, on the other hand, involves unnecessary code that can lead to mistakes and confusion [44]. Other methods often struggle to identify and address these underlying issues because they treat the code holistically. This can lead to confusion over bottom-level details when dealing with complex logical errors. By contrast, the hierarchical decomposition in MGDebugger allows it to focus on different levels of code granularity. This enables more effective identification and correction of bugs. These results demonstrate the robustness and versatility of MGDebugger across various bug types.

3.5 Performance Consistency (RQ4)

We further assess the versatility of MGDebugger in debugging code of varying lengths (i.e., number of tokens), since code length often correlates with complexity and debugging challenges. We categorize code snippets into short, medium, and long groups based on the 1/3 and 2/3 quantiles of code length distribution, ensuring equal sample sizes across the three categories. We subsequently analyze the repair success rate scores obtained by MGDebugger and baselines when using DeepSeek-Coder as the backbone LLM.

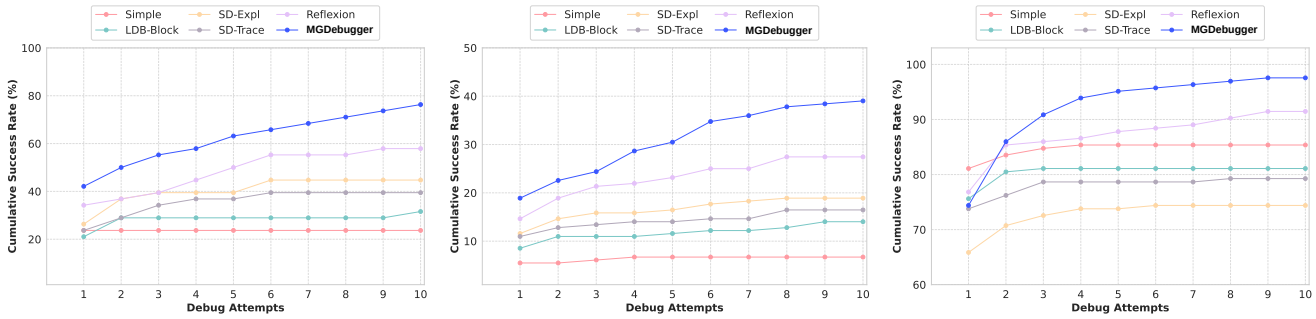
The results are presented in Figure 3. We observe that as the code length increases, most methods experience an obvious decrease in performance due to the increased complexity. Notably, MGDebugger consistently outperforms other methods across different code lengths on all three datasets. This performance advantage is particularly pronounced when debugging longer and more complex code snippets on HumanEval and HumanEvalFix, where MGDebugger maintains higher repair success rates compared to baseline methods.

Another important factor for LLM-based debugging is the number of debugging attempts. Iterative debugging retries allows LLMs to refine their corrections over multiple passes, potentially leading to better outcomes [65, 75]. We aim to assess MGDebugger's ability to improve over successive iterations. Following Zhong et al. [75], we vary the number of debugging attempts from 1 to 10 using DeepSeek-Coder on different dataset and collect the cumulative repair success rate (RSR) scores for each method. We clarify that a single iteration refers to one complete bottom-up traversal of



(a) HumanEval dataset (b) MBPP dataset (c) HumanEvalFix dataset

Figure 3: Repair success rate of different methods when debugging code of different lengths.



(a) HumanEval dataset (b) MBPP dataset (c) HumanEvalFix dataset

Figure 4: Impact of debug attempts on the cumulative repair success rate of different methods.

the function decomposition tree, including test case generation and simulated execution for all subfunctions. Even if a function is decomposed into multiple subfunctions, the process is counted as a single iteration if it involves one full pass from the leaves to the root.

The results in Figure 4 show that MGDebugger achieves the highest cumulative RSR score among all methods. As shown in Figure 4(a), MGDebugger demonstrates steady improvement across iterations on HumanEval dataset, effectively helping the model correct its own generation errors and reaching a significantly higher success rate compared to baselines. For MBPP in Figure 4(b), MGDebugger shows consistent gains with each attempt, highlighting its ability to systematically identify and fix bugs in LLM-generated code. Figure 4(c) shows that on HumanEvalFix, MGDebugger achieves over 95% accuracy after 5 attempts and continues to improve gradually to reach 97.6%, though the improvement rate slows due to a few particularly challenging cases requiring specialized handling.

These results demonstrate MGDebugger's strong potential in enhancing LLM-based code generation systems. By effectively debugging LLM-generated code through multiple iterations, MGDebugger not only improves the immediate code quality but also shows promise in strengthening the overall reliability of AI code generation systems. This iterative refinement capability is particularly valuable as it allows LLMs to learn from and correct their own mistakes, potentially leading to more robust and dependable automated programming solutions.

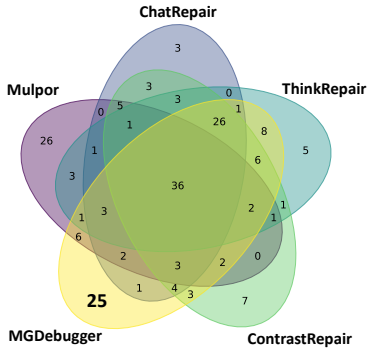


Figure 5: Bug fix Venn diagram in Defects4J V1.2.

3.6 Generalization to Real-World Software Defects (RQ5)

While our primary focus is to explore LLMs' ability to fix LLM-generated code, we also evaluate MGDebugger on repairing real-world, human-written bugs. We leverage the widely used Defects4J dataset [25], which contains real-world defects from open-source

Java projects. We compare MGDebugger with state-of-the-art program repair approaches, including ChatRepair [62], Mulpor [36], ContrastRepair [27], ThinkRepair [65], GAMMA [70], and CodexRepair [61]. Following prior APR tools [27, 36, 62, 65], we use the 255 single-function bugs in Defects4J v1.2 and 82 single-line bugs in Defects4J v2.0 for evaluation and provide the perfect fault localization for each bug. To ensure a fair comparison, we use gpt-3.5-turbo-0301 as the LLM backbone and set the maximum number of generated patches to 200 following prior works [27, 62]. For patch validation, we follow the established methodology from prior works [27, 36, 62, 65, 70] and conduct manual validation by three experienced engineers who independently assess each patch. A "correct" patch is defined as one that is semantically equivalent to the ground truth patch, while a "plausible" patch passes all available test cases but may not fix the underlying bug. The engineers discuss inconsistencies and reach consensus through detailed examination of patch semantics and equivalence to ground truth solutions.

Table 5 presents the number of bugs fixed by MGDebugger and various state-of-the-art program repair approaches across different projects in Defects4J. MGDebugger demonstrates superior overall performance, successfully fixing 129 bugs in Defects4J v1.2 and 64 bugs in Defects4J v2.0. This represents an improvement of 15 more bugs (13.2%) over ChatRepair on Defects4J v1.2 and 5 more bugs (8.5%) over Mulpor on Defects4J v2.0. Notably, MGDebugger outperforms all other approaches on 5 out of 6 individual projects, with particularly strong absolute improvements.

Figure 5 illustrates the overlap of bugs fixed by MGDebugger and four strong baseline approaches (using ChatRepair results from the replication package of ContrastRepair [27] due to unavailable bug id details in the original paper [62]). The diagram shows 36 bugs fixed by all methods, while MGDebugger uniquely fixes 25 distinct bugs, demonstrating its capability with complex defects. This complementarity with other approaches (e.g., Mulpor's 26 uniquely fixed bugs) suggests potential for combining methods to achieve even higher repair rates, as different debugging strategies appear effective for various bug types and code contexts.

Overall, these results demonstrate that MGDebugger generalizes well beyond debugging LLM-generated code to fixing real-world software defects in production systems. The hierarchical decomposition and systematic debugging approach of MGDebugger proves effective for human-written code with complex project structures and dependencies. This generalizability is particularly important as it suggests that MGDebugger could serve as a unified debugging framework applicable to both AI-generated and human-written code.

4 Discussion

4.1 Case Study

We perform a qualitative analysis of how MGDebugger effectively identifies and corrects buggy parts compared to baseline methods. Figure 6 shows an example of debugging code snippets from the HumanEvalFix dataset using MGDebugger and other representative methods, with DeepSeek-Coder-V2-Lite as the backbone LLM. The original buggy solution computes the Collatz sequence with an incorrect computation logic of $n = n \times 2 + 1$. While other methods correct the computation to $n = n \times 3 + 1$, they introduce a new bug that misses the last "1" in the Collatz sequence. This is possibly because they get distracted by the need to filter odd numbers, and thus move the operation of appending the number to the results before updating n. MGDebugger excelled by decomposing the problem into distinct subfunctions: sequence generation and odd number filtering. By debugging each subfunction independently, MGDebugger ensured comprehensive error correction, including the subtle requirement of incorporating 1 into the Collatz sequence. This approach demonstrates MGDebugger's ability to handle complex, multi-step problems more effectively than holistic debugging methods. Additionally, it highlights MGDebugger's ability to not only fix bugs but also restructure code for enhanced clarity and correctness, demonstrating its potential in improving the quality of LLM-generated code.

4.2 Individual Component Analysis

To further validate the quality of individual components, we conducted manual verification on 38 examples from HumanEval, including 29 successfully debugged and 9 failed cases, using DeepSeek-Coder-V2-Lite. Our analysis reveals three key findings: 1) **Code Decomposition**: All 38 decomposed versions are functionally equivalent to the original functions, demonstrating 100% semantic correctness in our decomposition process. Since the overall codes have been provided, it's less likely that the model includes new bugs when transforming the code into subfunctions, demonstrating the high quality of our LLM-based decomposition approach. 2) **Test Case Generation**: 291 out of 312 generated test cases, representing 93.2% accuracy, were correct when the model is instructed to strictly follow the public test cases when generating test cases for each subfunction. Since our approach derives test cases exclusively from the original public test cases, with a mean test coverage of 71.3%, mutation testing scores of 9.2% and test diversity of 28.6%, we maintain the inherent quality characteristics of the original test suite while enhancing debugging effectiveness. Additionally, in a supplementary experiment on 9 failed debugging cases, providing additional test cases that exposed errors enabled successful debugging of 2 additional functions, indicating potential for further improvement with enhanced test diversity and coverage. 3) **Simulated Execution**: The LLM correctly predicted 389 of 413 breakpoints, achieving 94.2% accuracy, with failures occurring only in cases requiring complex multi-step reasoning. The high accuracy aligns well with existing studies [45, 73]. Notably, when we manually corrected the prediction errors, 2 of the 9 failed debugging cases could be successfully fixed, indicating potential for further improvement with enhanced simulation accuracy. According to our manual analysis, the high quality of each component in MGDebugger contributes to the overall effectiveness, as demonstrated by the ablation results. This suggests our approach is robust and that further improvements in component quality could lead to even better results.

4.3 Bug Granularity Analysis

To validate our main claim that hierarchical debugging helps find bugs at different granularity levels, we manually analyzed the 23 problems successfully fixed by MGDebugger using DeepSeek-Coder

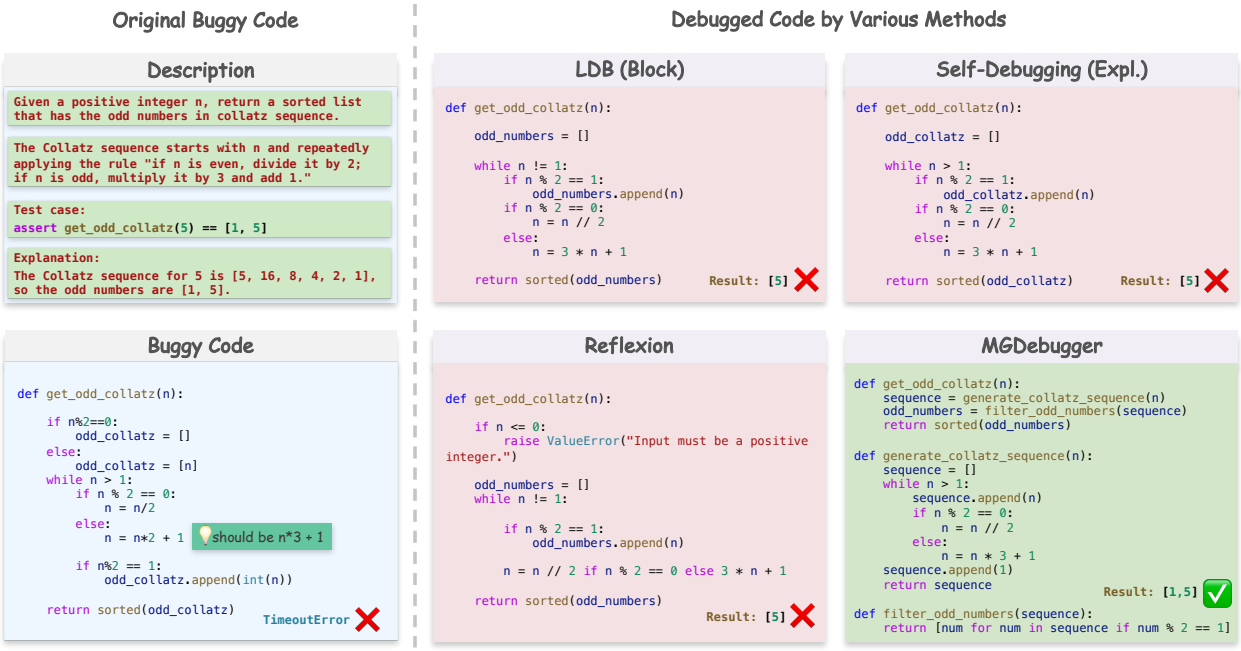


Figure 6: Examples of code debugging by various methods on HumanEvalFix with DeepSeek-Coder.

on HumanEval. Our analysis reveals that 13 out of 23 cases, representing 56.5%, involved low-level bugs such as syntactic errors, API misuse, incorrect operators, and variable name errors, while 10 out of 23 cases, representing 43.5%, involved high-level bugs including algorithmic logic errors, incorrect problem understanding, and missing core logic. This distribution demonstrates that MGDebugger effectively addresses both syntactic and semantic faults across different granularity levels, validating our hierarchical debugging strategy for handling diverse error types. The ability to handle both categories showcases MGDebugger's versatility across various debugging scenarios.

4.4 Evaluation on Unseen Data

Since HumanEval and Defects4J are potentially contained in training data of LLMs, potential data leakage could affect our evaluation. To mitigate this threat, we have employed several strategies: (1) we utilized the HumanEvalFix dataset, which consists of human-crafted bugs created after the training cutoff dates of Codestral; (2) our experimental design inherently reduces the impact of potential data leakage by focusing exclusively on test cases where the No-Debugging LLM fails to produce correct solutions [9, 47]. If data leakage had a strong effect for a particular case, the model would likely have generated a correct solution on its first attempt, and such cases would not be included in our debugging evaluation set.

To further address potential data leakage concerns, we conducted additional experiments on LiveCodeBench [20], which contains programming problems released after LLM training cutoffs. Using 131 problems from the February-May 2025 evaluation window with GPT-4o, MGDebugger achieves 36.6% accuracy with 10.8% repair success rate, outperforming all baselines by +7.6% over No-Debugging and +3.0% over Reflexion (Table 6). These results also confirm that MGDebugger's improvements are not artifacts of data leakage but represent genuine debugging capabilities on completely unseen data.

Table 6: Results on LiveCodeBench with GPT-4o.

Method	Acc. (%)	RSR (%)
No-Debugging	29.0	–
Self-Debugging (Expl.)	32.8 (+3.8)	4.3
Reflexion	33.6 (+4.6)	5.4
MGDebugger	**36.6 (+7.6)**	**10.8**

4.5 Threats to Validity

While our evaluation is comprehensive, several threats to validity should be acknowledged. 1) The first is the risk of limited generalizability across programming languages and domains. To mitigate this concern, we have evaluated MGDebugger across four distinct benchmarks spanning both Python and Java languages. This comprehensive validation across different language contexts strengthens our confidence in the approach's broad applicability. 2) Our method's performance may vary with different LLM architectures. To address this concern, we conducted experiments with various open-source LLMs of different sizes as well as several commercial models. The consistent improvements observed across this diverse set of models suggest that MGDebugger's benefits are robust across different model architectures and capabilities. 3) For our evaluation on Defects4J, an internal threat arises from the manual validation process used to determine the correctness of the generated patches. To mitigate this concern, we conducted thorough examinations of each patch, following established validation practices from prior work in automated program repair [27, 36, 62, 65, 70].

Despite these potential limitations, our extensive evaluation across multiple dimensions—including various models, datasets,

bug types, and code complexities—provides strong evidence that MGDebugger represents a significant advancement in debugging LLM-generated codes. The consistency of our results and the magnitude of improvement over existing methods indicate broad applicability and robust performance that is likely to extend to many real-world debugging scenarios.

5 Related Work

5.1 Large Language Models for Code

Recent models such as GPT4 [48], Codestral [42], and DeepSeek-Coder [1] have advanced code generation through instruction tuning and reinforcement learning from human feedback (RLHF) with mixed code and natural language data [2, 49, 77]. Code generation with LLMs has been enhanced by various techniques. Some approaches focus on improving the quality of generated code using planning algorithms, transitioning from outlines to detailed implementations [66, 71, 74]. Other methods sample multiple programs from the same LLM and rank them to identify the best one [6, 8, 46]. Additionally, some works leverage multi-agent collaboration frameworks to enhance code generation quality [11, 17, 68]. These approaches aim to optimize the production of correct code from the outset. By contrast, MGDebugger targets the post-generation phase, focusing on debugging and fixing errors that inevitably arise during the code generation process.

5.2 Repairing LLM-Generated Code

Program repair is a critical aspect of software development, aiming to automatically identify and fix bugs in code [13, 25, 51, 64]. There are two main streams of research in repairing code generated by LLMs: (1) training models to repair code [10, 18, 22, 28, 43, 73] and (2) providing external feedback to the raw pretrained models to fix code [9, 16, 24, 47, 75]. By contrast to previous work that trains separate models for code repair [10, 43, 73], MGDebugger does not require task-specific retraining but takes advantage of the inherent capabilities of pretrained LLMs. This flexibility allows MGDebugger to operate in zero-shot settings, offering a lightweight and scalable alternative. Furthermore, exploring the ability of LLMs to fix their own code is a promising direction for self-improvement training of the LLMs [5].

MGDebugger falls under the category of work that leverages pretrained models to fix code by reasoning with external feedback. Several recent methods [4, 32, 47, 69] utilize execution results from test cases to guide LLMs in code correction. More recent works have explored advanced debugging techniques utilizing LLM's reasoning ability. Reflexion [53] prompts LLMs to reflect on the generated code and uses a memory buffer for iterative refinement. Self-Debugging [9] prompts LLMs to explain or dry run generated programs, known as rubber duck debugging. LDB [75] segments programs into basic blocks, tracking variable values during runtime after each block to verify the correctness against the task description. Although these methods incorporate detailed execution feedback and iterative refinement, they treat the whole function as a single unit and perform sequential debugging, limiting their effectiveness with complex code [9, 15]. MGDebugger addresses this issue by introducing a hierarchical approach, debugging from low-level errors to high-level flaws. This method ensures a more systematic and accurate debugging process, especially for complex and multifunctional systems.

5.3 Automated Program Repair

Traditional APR techniques can be broadly classified into three categories: template-based [39, 70], heuristic-based [26, 31] and constraint based [30, 41, 76]. While these approaches have demonstrated effectiveness, they often face challenges in scalability and patch diversity. The advent of deep learning has shifted the focus to learning-based approaches, particularly those framing program repair as a neural machine translation task [23, 35, 36, 40]. These methods use encoder-decoder architectures to translate buggy code into correct versions, achieving notable success but remaining limited by the quality and quantity of training data [65]. More recently, LLM-based APR tools, powered by models like ChatGPT and Codex, have achieved superior fix rates on standard benchmarks like Defects4J [25] by generating diverse and contextually relevant patches [15, 27, 61–63, 65, 72].

MGDebugger differs from existing APR approaches in both objective and methodology. MGDebugger focuses on enabling LLMs to correct their own mistakes from code generation, working towards more reliable AI code generation systems. Methodologically, MGDebugger introduces a novel bottom-up hierarchical debugging strategy combined with test case generation and LLM-simulated execution, unleashing the potential of LLMs' reasoning capabilities.

6 Conclusion

In this paper, we introduced MGDebugger, a novel hierarchical code debugging framework that systematically fixes bugs at multiple levels of granularity. By decomposing complex code into a hierarchical structure, generating targeted test cases and employing LLM-simulated execution, MGDebugger effectively identifies and fixes bugs ranging from syntax errors to logical flaws in a bottom-up manner. Experiments across various models and datasets demonstrate MGDebugger's superior performance over existing methods, particularly in handling complex logical errors and longer code snippets.

Future work can build upon this foundation to develop more advanced code generation and debugging methodologies. One direction is to extend MGDebugger to handle more complex bugs and code structures, such as multi-file projects and codebase with multiple dependencies. Another direction is to explore the collaboration of hierarchical code generation approaches with hierarchical debugging, enabling end-to-end code generation and debugging systems. Furthermore, integrating MGDebugger into self-training systems to correct outputs from base models, then retraining the base models with the corrected data, could potentially improve their performance iteratively.

Acknowledgments

This research is funded by the National Key Research and Development Program of China (Grant No. 2023YFB4503802) and the Natural Science Foundation of Shanghai (Grant No. 25ZR1401175). The authors would like to thank National Supercomputer Center in Guangzhou for providing high performance computational resources.

References

[1] DeepSeek AI. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence. arXiv:2406.11931

[2] Anthropic. 2024. Introducing Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs] doi:10.48550/arXiv.2108.07732

[4] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. arXiv:2403.17134 [cs] doi:10.48550/arXiv.2403.17134

[5] Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, Ilya Sutskever, and Jeff Wu. 2023. Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision. arXiv:2312.09390 doi:10.48550/arXiv.2312.09390

[6] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. In *The Twelfth International Conference on Learning Representations*.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374

[8] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2023. Universal Self-Consistency for Large Language Model Generation. arXiv:2311.17311

[9] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*.

[10] Yangruibo Ding, Marcus J. Min, Gail Kaiser, and Baishakhi Ray. 2024. CYCLE: Learning to Self-Refine the Code Generation. *Proc. ACM Program. Lang.* 8, OOPSLA1 (April 2024), 108:392–108:418. doi:10.1145/3649825

[11] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Trans. Softw. Eng. Methodol.* (June 2024). doi:10.1145/3672459

[12] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, Yan Liu, Enyu Zhou, Ming Zhang, Yuhao Zhou, Yueming Wu, Rui Zheng, Ming Wen, Rongxiang Weng, Jingang Wang, Xunliang Cai, Tao Gui, Xipeng Qiu, Qi Zhang, and Xuanjing Huang. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. arXiv:2407.06153

[13] Kavi Gupta, Peter Ebert Christensen, Xinyun Chen, and Dawn Song. 2020. Synthesize, Execute and Debug: Learning to Repair for Neural Program Synthesis. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 17685–17695.

[14] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Thirty-Fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

[15] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. 2024. A Deep Dive into Large Language Models for Automated Bug Localization and Repair. *Proc. ACM Softw. Eng.* 1, FSE (July 2024), 66:1471–66:1493. doi:10.1145/3660773

[16] Xueyu Hu, Kun Kuang, Jiankai Sun, Hongxia Yang, and Fei Wu. 2024. Leveraging Print Debugging to Improve Code Generation in Large Language Models. arXiv:2401.05319

[17] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 doi:10.48550/arXiv.2312.13010

[18] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An Empirical Study on Fine-Tuning Large Language Models of Code for Automated Program Repair. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Luxembourg, Luxembourg, 1162–1174. doi:10.1109/ASE56229.2023.00181

[19] Ayaz Isazadeh, Habib Izadkhah, and Islam Elgedawy. 2017. *Source Code Modularization*. Springer International Publishing, Cham. doi:10.1007/978-3-319-63346-6

[20] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations*.

[21] Naman Jain, Tianjun Zhang, Wei-Lin Chiang, Joseph E. Gonzalez, Koushik Sen, and Ion Stoica. 2023. LLM-Assisted Code Cleaning For Training Accurate Code Generators. arXiv:2311.14904 doi:10.48550/arXiv.2311.14904

[22] Nan Jiang, Xiaopeng Li, Shiqi Wang, Qiang Zhou, Soneya Binta Hossain, Baishakhi Ray, Varun Kumar, Xiaofei Ma, and Anoop Deoras. 2024. Training LLMs to Better Self-Debug and Explain Code. In *NeurIPS2024*.

[23] Nan Jiang, Thibaud Lutellier, Yiling Lou, Lin Tan, Dan Goldwasser, and Xiangyu Zhang. 2023. KNOD: Domain Knowledge Distilled Tree Decoder for Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, Melbourne, Victoria, Australia, 1251–1263. doi:10.1109/ICSE48619.2023.00111

[24] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. arXiv:2306.02907 [cs] doi:10.48550/arXiv.2306.02907

[25] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055

[26] YoungJae Kim, Yechan Park, Seungheon Han, and Jooyong Yi. 2024. Enhancing the Efficiency of Automated Program Repair via Greybox Analysis. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 1719–1731. doi:10.1145/3691620.3695602

[27] Jiaolong Kong, Xiaofei Xie, Mingfei Cheng, Shangqing Liu, Xiaoning Du, and Qi Guo. 2025. ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs. *ACM Trans. Softw. Eng. Methodol.* (March 2025). doi:10.1145/3719345

[28] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D. Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, Lei M. Zhang, Kay McKinney, Disha Shrivastava, Cosmin Paduraru, George Tucker, Doina Precup, Feryal Behbahani, and Aleksandra Faust. 2024. Training Language Models to Self-Correct via Reinforcement Learning. arXiv:2409.12917 [cs]

[29] Emanuele La Malfa, Christoph Weinhuber, Orazio Torre, Fangru Lin, Samuele Marro, Anthony Cohn, Nigel Shadbolt, and Michael Wooldridge. 2024. Code simulation challenges for large language models. *arXiv preprint arXiv:2401.09074* (2024).

[30] Xuan-Bach D. Le, Duc-Hiep Chu, David Lo, Claire Le Goues, and Willem Visser. 2017. S3: Syntax- and Semantic-Guided Repair Synthesis via Programming by Examples. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 593–604. doi:10.1145/3106237.3106309

[31] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Trans. Softw. Eng.* 38, 1 (Jan. 2012), 54–72. doi:10.1109/TSE.2011.104

[32] Cheryl Lee, Chunqiu Steven Xia, Jen-tse Huang, Zhouruixin Zhu, Lingming Zhang, and Michael R. Lyu. 2024. A Unified Debugging Approach via LLM-Based Multi-Agent Synergy. arXiv:2404.17153

[33] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2023. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. arXiv:2312.04474

[34] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158

[35] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. DLFix: Context-Based Code Transformation Learning for Automated Program Repair. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 602–614. doi:10.1145/3377811.3380345

[36] Bo Lin, Shangwen Wang, Ming Wen, Liqian Chen, and Xiaoguang Mao. 2024. One Size Does Not Fit All: Multi-granularity Patch Generation for Better Automated Program Repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1554–1566. doi:10.1145/3650212.3680381

[37] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. arXiv:2305.01210 [cs] doi:10.48550/arXiv.2305.01210

[38] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Yang Wei, and Deheng Ye. 2023. RLTF: Reinforcement Learning from Unit Test Feedback. *Transactions on Machine Learning Research* (July 2023).

[39] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawende F. Bissyandè. 2019. AVATAR: Fixing Semantic Bugs with Fix Patterns of Static Analysis Violations. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE Computer Society, 1–12. doi:10.1109/SANER.2019.8667970

[40] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2020)*. Association for Computing Machinery, New York, NY, USA, 101–114. doi:10.1145/3395363.3397369

[41] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. Association for Computing Machinery, New York, NY, USA, 691–701. doi:10.1145/2884781.2884807

[42] Mistral AI team. 2024. Codestral: Hello, World! https://mistral.ai/news/codestral/.

[43] Seungjun Moon, Hyungjoo Chae, Yongho Song, Taeyoon Kwon, Dongjin Kang, Kai Tzu-iunn Ong, Seung-won Hwang, and Jinyoung Yeo. 2024. Coffee: Boost Your Code LLMs by Fixing Bugs with Feedback. arXiv:2311.07215 [cs]

[44] Niklas Muennighoff, Qian Liu, Armel Randy Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, and Shayne Longpre. 2023. OctoPack: Instruction Tuning Code Large Language Models. In *The Twelfth International Conference on Learning Representations*.

[45] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. NExT: Teaching Large Language Models to Reason about Code Execution. arXiv:2404.14662

[46] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. arXiv:2302.08468 doi:10.48550/arXiv.2302.08468

[47] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2023. Is Self-Repair a Silver Bullet for Code Generation?. In *The Twelfth International Conference on Learning Representations*.

[48] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs] doi:10.48550/arXiv.2303.08774

[49] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model Is Secretly a Reward Model. In *Thirty-Seventh Conference on Neural Information Processing Systems*.

[50] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (Jan. 2024), 85–105. doi:10.1109/TSE.2023.3334955

[51] Yuchen Shao, Yuheng Huang, Jiawei Shen, Lei Ma, Ting Su, and Chengcheng Wan. 2025. Are LLMs Correctly Integrated into Software Systems?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1178–1190.

[52] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. 2024. Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers. In 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). *IEEE Computer Society* (2024), 51–62.

[53] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Thirty-Seventh Conference on Neural Information Processing Systems*.

[54] Qwen Team. 2023. Qwen Technical Report. arXiv:2309.16609 [cs] doi:10.48550/arXiv.2309.16609

[55] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating Debugging Capability of Large Language Models. arXiv:2401.04621 doi:10.48550/arXiv.2401.04621

[56] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs] doi:10.48550/arXiv.2302.13971

[57] Changjie Wang, Mariano Scazzariello, Anoud Alshnakat, Roberto Guanciale, Dejan Kostić, and Marco Chiesa. 2025. Dissect-and-Restore: AI-based Code Verification with Transient Refactoring. *arXiv preprint arXiv:2510.25406* (2025).

[58] Song Wang, Nishtha Shrestha, Abarna Kucheri Subburaman, Junjie Wang, Moshi Wei, and Nachiappan Nagappan. 2021. Automatic Unit Test Generation for Machine Learning Libraries: How Far Are We?. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1548–1560. doi:10.1109/ICSE43902.2021.00138

[59] Jiaxin Wen, Ruiqi Zhong, Pei Ke, Zhihong Shao, Hongning Wang, and Minlie Huang. 2024. Learning Task Decomposition to Assist Humans in Competitive Programming. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 11700–11723. doi:10.18653/v1/2024.acl-long.629

[60] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen. 1981. The Effect of Modularization and Comments on Program Comprehension. In *Proceedings of the 5th International Conference on Software Engineering (ICSE '81)*. IEEE Press, San Diego, California, USA, 215–223.

[61] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1482–1494. doi:10.1109/ICSE48619.2023.00129

[62] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each Using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Vienna Austria, 819–831. doi:10.1145/3650212.3680323

[63] Qi Xin, Haojun Wu, Jinran Tang, Xinyu Liu, Steven P. Reiss, and Jifeng Xuan. 2024. Detecting, Creating, Repairing, and Understanding Indivisible Multi-Hunk Bugs. *Reproduction Package for Article Detecting, Creating, Repairing, and Understanding Indivisible Multi-Hunk Bugs* 1, FSE (July 2024), 121:2747–121:2770. doi:10.1145/3660828

[64] Michihiro Yasunaga and Percy Liang. 2021. Break-It-Fix-It: Unsupervised Learning for Program Repair. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 11941–11952.

[65] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. ThinkRepair: Self-Directed Automated Program Repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1274–1286. doi:10.1145/3650212.3680359

[66] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah Goodman, and Nick Haber. 2023. Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions. In *Thirty-Seventh Conference on Neural Information Processing Systems*.

[67] Andreas Zeller. 2009. *Why Programs Fail: A Guide to Systematic Debugging*. Morgan Kaufmann.

[68] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. arXiv:2401.07339

[69] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 769–787. doi:10.18653/v1/2023.acl-long.45

[70] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2024. Gamma: Revisiting Template-based Automated Program Repair via Mask Prediction. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE Press, Echternach, Luxembourg, 535–547. doi:10.1109/ASE56229.2023.00063

[71] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2022. Planning with Large Language Models for Code Generation. In *The Eleventh International Conference on Learning Representations*.

[72] Yuwei Zhang, Zhi Jin, Ying Xing, Ge Li, Fang Liu, Jiaxin Zhu, Wensheng Dou, and Jun Wei. 2025. PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing. *ACM Trans. Softw. Eng. Methodol.* (Feb. 2025). doi:10.1145/3718739

[73] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. 2024. OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement. arXiv:2402.14658 doi:10.48550/arXiv.2402.14658

[74] Wenqing Zheng, S. P. Sharan, Ajay Kumar Jaiswal, Kevin Wang, Yihan Xi, Dejia Xu, and Zhangyang Wang. 2023. Outline, Then Details: Syntactically Guided Coarse-To-Fine Code Generation. In *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 42403–42419.

[75] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step. In *Findings of the Association for Computational Linguistics ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand and virtual meeting, 851–870.

[76] Qihao Zhu, Zeyu Sun, Wenjie Zhang, Yingfei Xiong, and Lu Zhang. 2023. Tare: Type-Aware Neural Program Repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1443–1455. doi:10.1109/ICSE48619.2023.00126

[77] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2020. Fine-Tuning Language Models from Human Preferences. arXiv:1909.08593